

NAME : VELMURUGAN A

LAB EXPERIMENT : LESSON 2

EXPERIMENT: 1 AND 2

CHAPTER -2 CODES

[1]

✓ 15s

```
arr = eval(input("Enter list: "))

n = len(arr)

for i in range(n):
    for j in range(i + 1, n):
        if arr[i] > arr[j]:
            arr[i], arr[j] = arr[j], arr[i]

print("Sorted List:", arr)
```

▼

Enter list: []
Sorted List: []

[2]

✓ 15s



```
arr = list(map(int, input("Enter elements: ").split()))

n = len(arr)

for i in range(n):
    min_index = i

    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted Array:", arr)
```

▼

... Enter elements: 5 2 9 1 5 6
Sorted Array: [1, 2, 5, 5, 6, 9]

EXPERIMENT: 3 AND 4

[3]
✓ 17s

```
arr = list(map(int, input("Enter elements: ").split()))

n = len(arr)

for i in range(n):
    swapped = False

    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swapped = True

    if not swapped:
        break

print("Sorted Array:", arr)
```

▼ Enter elements: 64 25 12 22 11
Sorted Array: [11, 12, 22, 25, 64]

[4]
✓ 10s

```
arr = list(map(int, input("Enter elements: ").split()))

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted Array:", arr)
```

▼ ... Enter elements: 3 1 4 1 5 9 2 6 5 3
Sorted Array: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

EXPERIMENT: 5 AND 6

[5]

✓ 20s

```
arr = list(map(int, input("Enter sorted array: ").split()))
k = int(input("Enter k: "))

missing = 0
num = 1
i = 0

while True:
    if i < len(arr) and arr[i] == num:
        i += 1
    else:
        missing += 1
        if missing == k:
            print("Kth Missing Number:", num)
            break
        num += 1
```

▼

Enter sorted array: 2 3 4 7 11
Enter k: 5
Kth Missing Number: 9

[6]

✓ 9s



```
nums = list(map(int, input("Enter array: ").split()))

n = len(nums)

for i in range(n):
    left = (i == 0 or nums[i] > nums[i-1])
    right = (i == n-1 or nums[i] > nums[i+1])

    if left and right:
        print("Peak Index:", i)
        break
```

▼

... Enter array: 1 2 3 1
Peak Index: 2

EXPERIMENT: 7 AND 8

[7]

✓ 7s

```
haystack = input("Enter main string: ")
needle = input("Enter substring: ")

index = haystack.find(needle)

print("Index:", index)
```

▼

```
Enter main string: sadbutsad
Enter substring: sub
Index: -1
```

[8]

✓ 2s

```
words = input("Enter words: ").split()

result = []

for i in range(len(words)):
    for j in range(len(words)):
        if i != j and words[i] in words[j]:
            result.append(words[i])
            break

print("Substrings:", result)
```

▼

```
Enter words: mass as hero superhero
Substrings: ['as', 'hero']
```

EXPERIMENT: 9 AND 10

```
import math
points = [(1,2), (4,5), (7,8), (3,1)]
min_dist = float('inf')
pair = ()
for i in range(len(points)):
    for j in range(i+1, len(points)):
        d = math.sqrt((points[i][0]-points[j][0])**2 +
                      (points[i][1]-points[j][1])**2)
        if d < min_dist:
            min_dist = d
            pair = (points[i], points[j])
print("Closest Pair:", pair)
print("Minimum Distance:", min_dist)
```

```
... Closest Pair: ((1, 2), (3, 1))
Minimum Distance: 2.23606797749979
```

```
import math
def distance(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
points = [(10,0), (11,5), (5,3), (9,3.5),
          (15,3), (12.5,7), (6,6.5), (7.5,4.5)]
min_dist = float('inf')
closest = ()
for i in range(len(points)):
    for j in range(i+1, len(points)):
        d = distance(points[i], points[j])

        if d < min_dist:
            min_dist = d
            closest = (points[i], points[j])
print("Closest Pair:", closest)
print("Minimum Distance:", min_dist)
```

```
Closest Pair: ((9, 3.5), (7.5, 4.5))
Minimum Distance: 1.8027756377319946
```

EXPERIMENT: 11 AND 12

```
from scipy.spatial import ConvexHull
points = [
    (1,1),
    (4,6),
    (8,1),
    (0,0),
    (3,3)
]
hull = ConvexHull(points)
print("Convex Hull Points:")
for i in hull.vertices:
    print(points[i])
```

```
... Convex Hull Points:
(0, 0)
(8, 1)
(4, 6)
```

```
from itertools import permutations
from math import sqrt
def distance(c1, c2):
    return sqrt((c1[0]-c2[0])**2 + (c1[1]-c2[1])**2)
cities = [(1,2), (4,5), (7,1), (3,6)]
start = cities[0]
min_distance = float('inf')
best_path = None
for path in permutations(cities[1:]):
    route = [start] + list(path) + [start]
    total = 0
    for i in range(len(route)-1):
        total += distance(route[i], route[i+1])
    if total < min_distance:
        min_distance = total
        best_path = route
print("Shortest Distance:", round(min_distance,2))
print("Shortest Path:", best_path)
```

```
Shortest Distance: 16.97
```


EXPERIMENT: 13

[14]

✓ 0s



```
from itertools import permutations
```

```
cost = [  
    [3,10,7],  
    [8,5,12],  
    [4,6,9]  
]
```

```
n = len(cost)
```

```
min_cost = float('inf')  
best = None
```

```
for assign in permutations(range(n)):  
    total = 0
```

```
    for worker in range(n):  
        total += cost[worker][assign[worker]]
```

```
    if total < min_cost:  
        min_cost = total  
        best = assign
```

```
print("Optimal Assignment:")
```

```
for worker in range(n):  
    print("Worker", worker+1, "-> Task", best[worker]+1)
```

```
print("Minimum Cost:", min_cost)
```



```
... Optimal Assignment:  
Worker 1 -> Task 3  
Worker 2 -> Task 2  
Worker 3 -> Task 1  
Minimum Cost: 16
```

EXPERIMENT: 14

```
15] from itertools import combinations
✓ 0s

weights = [2,3,1]
values = [4,5,3]
capacity = 4

n = len(weights)

best_value = 0
best_items = []

for r in range(n+1):
    for subset in combinations(range(n), r):

        total_weight = 0
        total_value = 0

        for i in subset:
            total_weight += weights[i]
            total_value += values[i]

        if total_weight <= capacity and total_value > best_value:
            best_value = total_value
            best_items = subset

print("Selected Items:", list(best_items))
print("Maximum Value:", best_value)
```

Selected Items: [1, 2]
Maximum Value: 8